Fig 1 - Battle Tanks: Top-Level System Block Diagram (FPGA fabric)

RV32IM 3-stage CPU runs the game rules in C; everything per-pixel / per-cycle / per-deadline is hardware. Top-level view only; gate-level detail is in Figs 2-10.

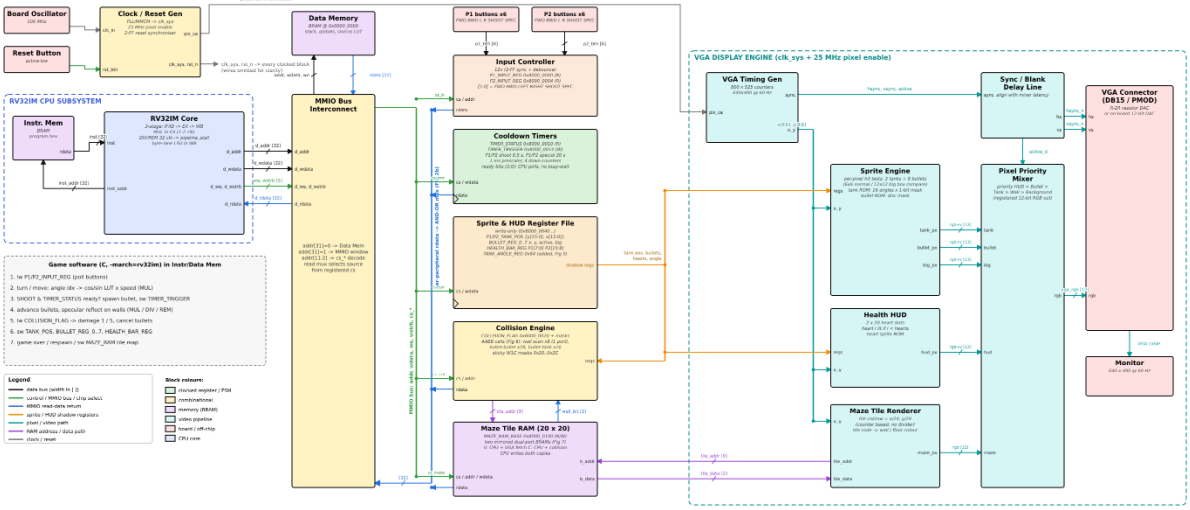


Fig 2a - MMIO Interconnect (1/2): Address Decode and Write Enables

Pure combinational. addr[31]=1 selects the MMIO window; addr[11:3] picks a peripheral; each write enable = cs & d_we

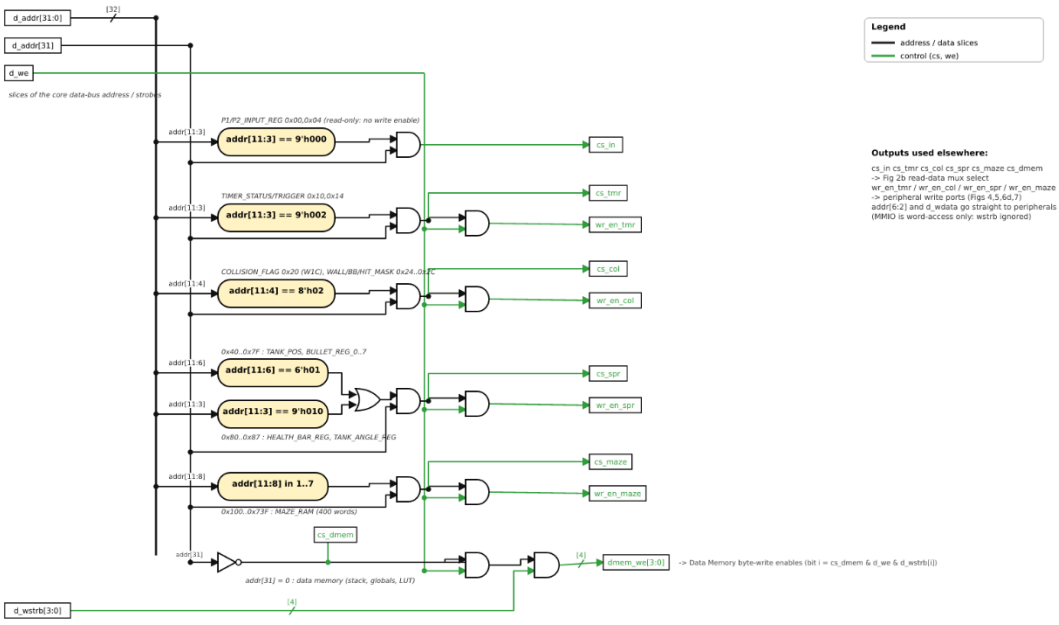


Fig 2b - MMIO Interconnect (2/2): Read-Data Path (registered select, AND-OR mux)

No shared / tri-state bus: every peripheral drives its own rdata; one-hot registered chip-selects gate them into d_rdata.

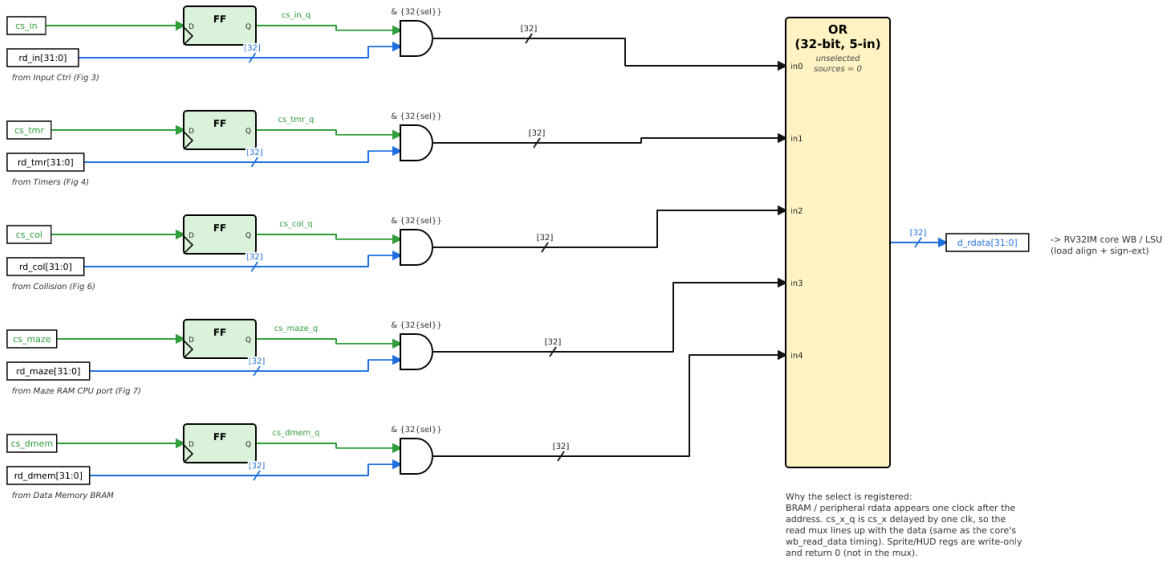
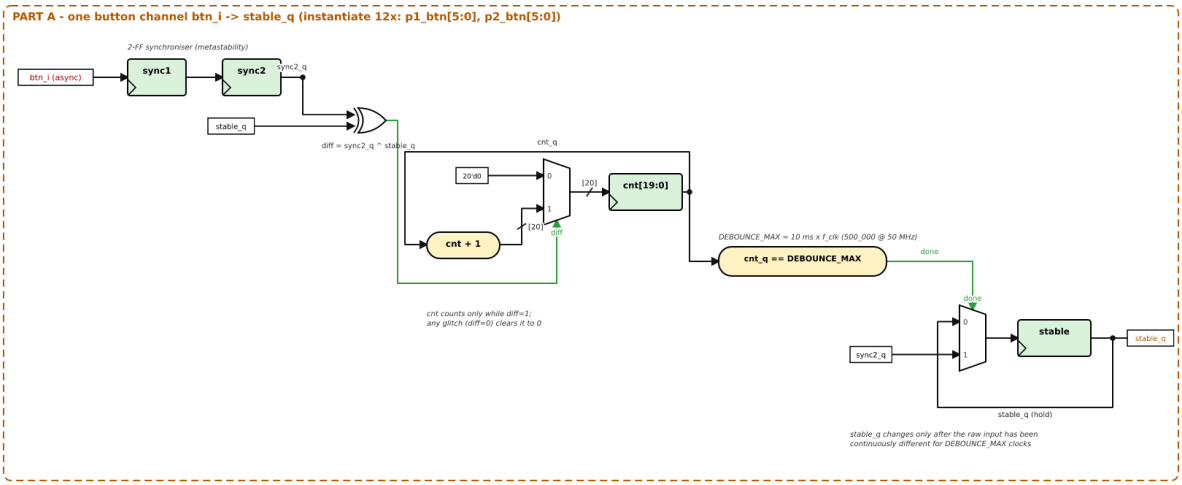


Fig 3 - Input Controller: 2-FF Synchroniser + Debouncer (x12) and MMIO Input Registers

Part A: one button channel (replicated 12x). Part B: stable bits packed into P1_INPUT_REG / P2_INPUT_REG and read through a 2:1 mux. All FFs: sync reset to 0 on rst_n.



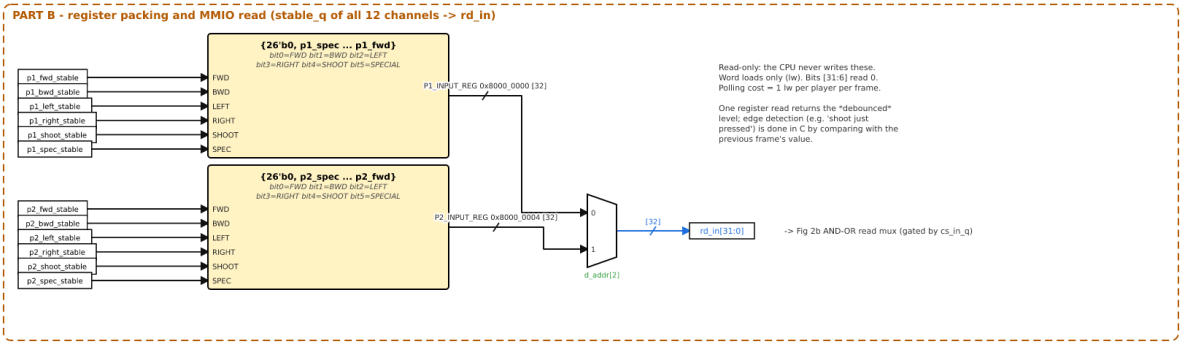
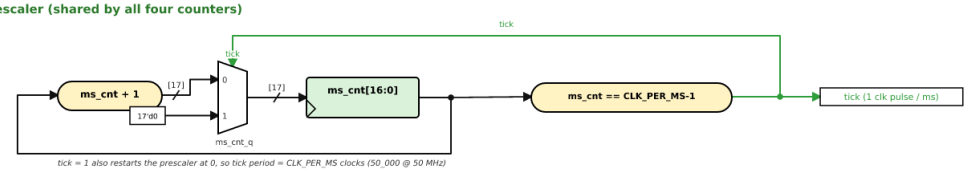


Fig 4 - Hardware Cooldown Timers (TIMER_STATUS 0x8000_0010 / TIMER_TRIGGER 0x8000_0014)

Part A: shared 1 ms tick. Part B: one down-counter (x4). Part C: trigger decode + status read. All registers: sync reset to 0.

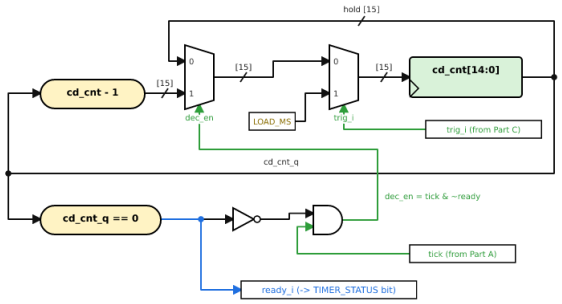
PART A - 1 ms prescaler (shared by all four counters)



tick = 1 also restarts the prescaler at 0, so tick period = CLK_PER_MS clocks (50_000 @ 50 MHz)

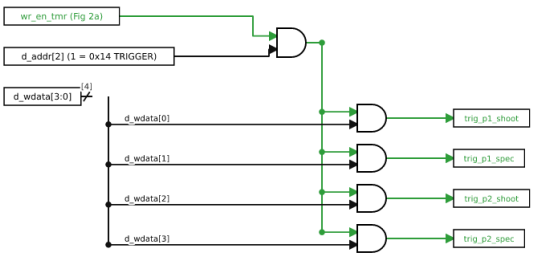
PART B - one cooldown channel (instantiate 4x: P1 shoot, P1 special, P2 shoot, P2 special)

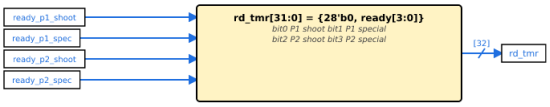
Behaviour per channel:
trig_i = 1 : reload cd_cnt = LOAD_MS (restart cooldown)
tick & cd_cnt != 0 : cd_cnt - 1 (counts down 1 ms)
cd_cnt == 0 : ready = 1, counter holds
(reset value 0 -> ready after reset)



LOAD_MS (per channel):
shoot = 500 (0.5 s)
special = 20000 (20 s)

PART C - MMIO write decode (TIMER_TRIGGER) and read (TIMER_STATUS)





trig_p1_shoot -> channel 0 (trig_i)
trig_p1_spec -> channel 1
trig_p2_shoot -> channel 2
trig_p2_spec -> channel 3

-> Fig 2b read mux (cs_tmr_q). C code: if (TIMER_STATUS & 1) fire();
no busy-wait loop; the CPU only samples ready bits once per frame.

Fig 5 - Sprite & HUD Register File (12 x 32-bit write-only MMIO registers)

Each register: comparator on d_addr[7:2] AND wr_en_spr -> select of a hold mux -> FF. Same row structure repeated 12 times; outputs run continuously to the Sprite Engine, Health HUD and Collision Engine.

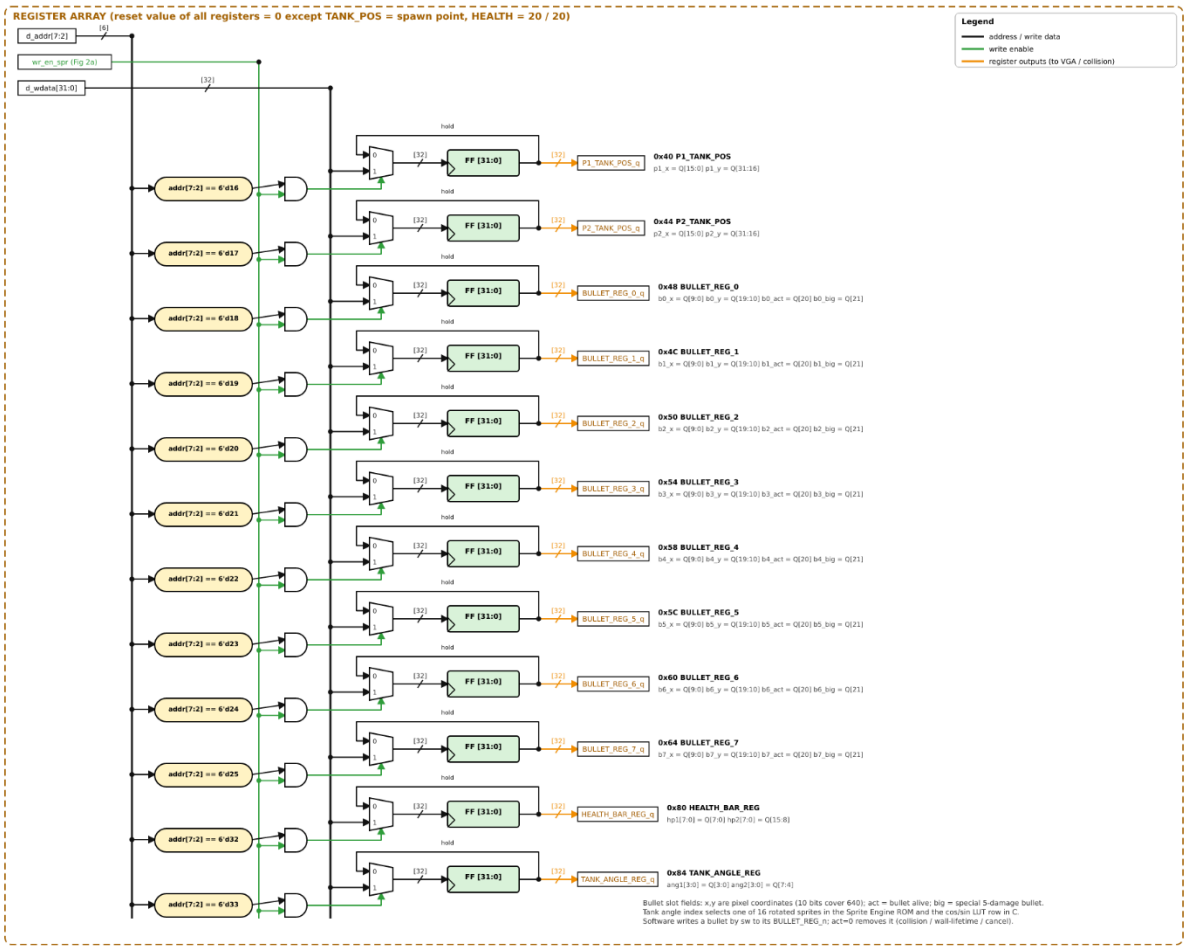


Fig 6a - Collision Engine: Axis-Aligned Bounding-Box (AABB) Overlap Cell

The one combinational cell replicated 28x (bullet-bullet) and 16x (bullet-tank). A and B overlap <=> they overlap on X AND on Y AND both are alive.

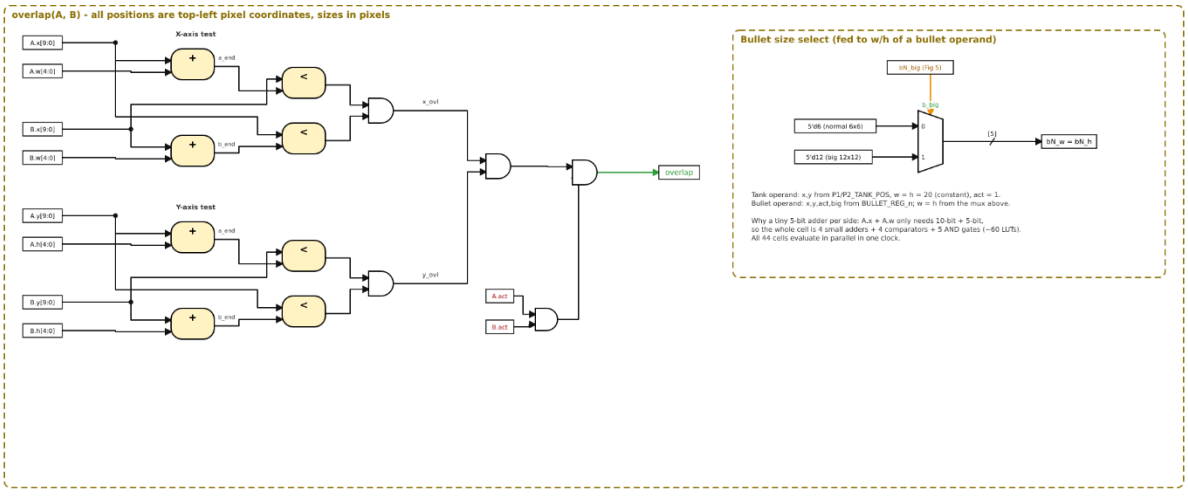


Fig 6b - Collision Engine (1/3): Bullet-Wall Scan (one shared maze-RAM lookup, 8 bullets in 8 clocks)

A scan counter time-shares ONE maze-RAM read port across the 8 bullet slots. tile_addr is registered and BRAM read takes 1 clk, so scan index and act are delayed 2 clocks to match.

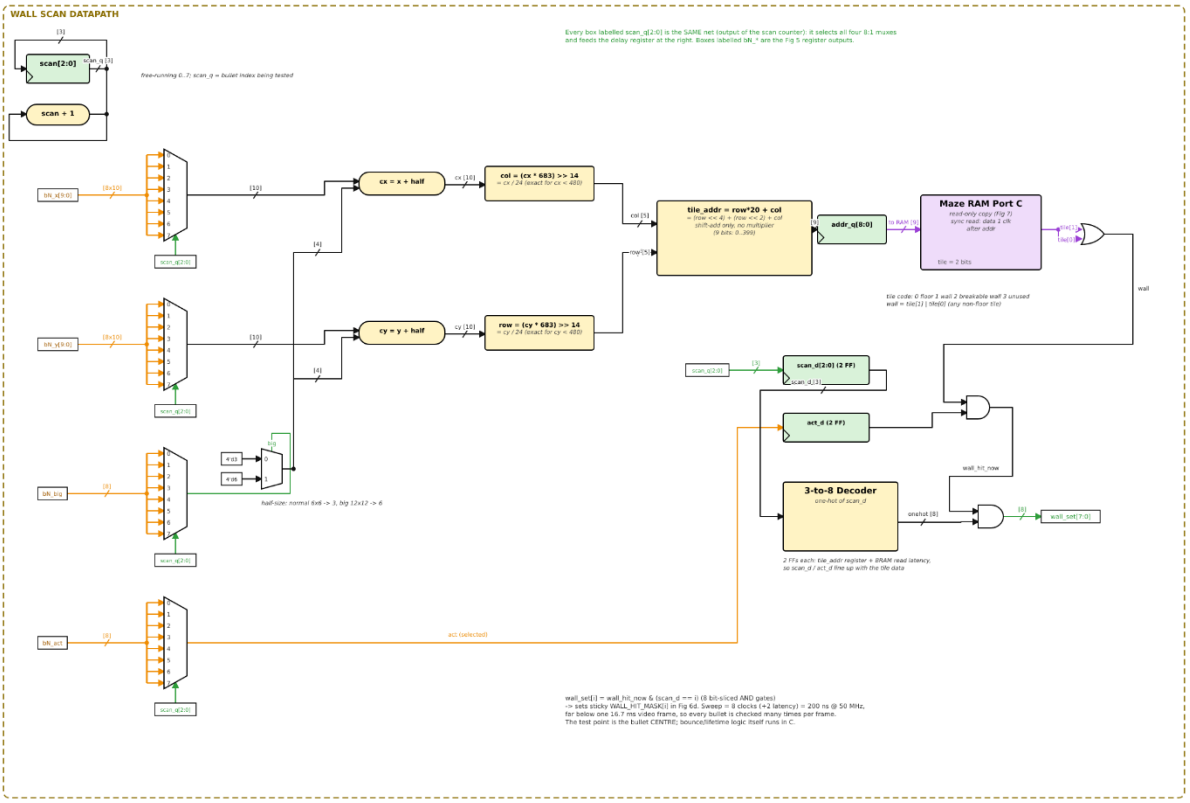
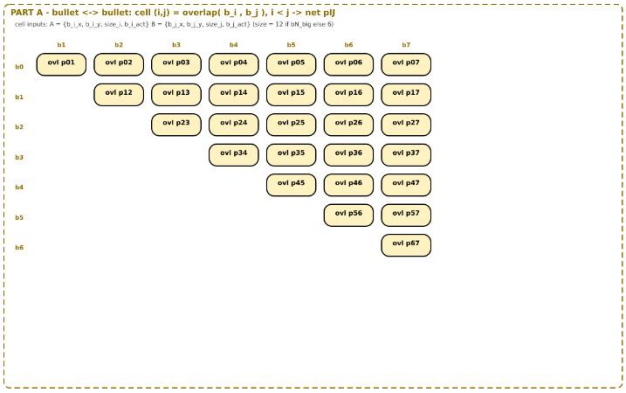
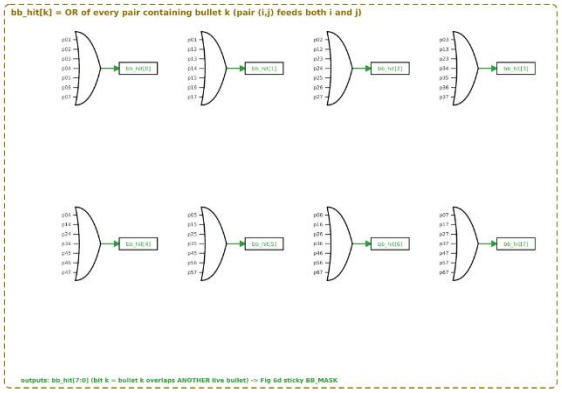
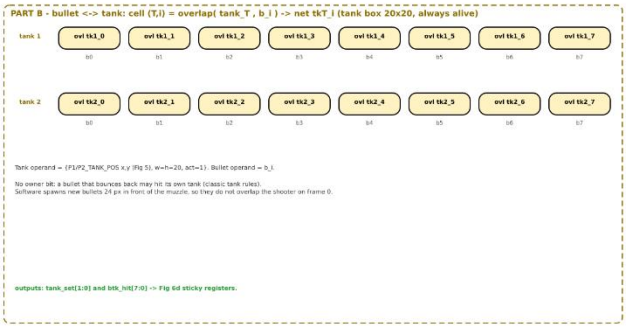


Fig 6c - Collision Engine (2/3): Bullet-Bullet Pair Matrix (28 cells) and Bullet-Tank Cells (16 cells)

Every cell is the Fig-6a overlap cell. All 44 evaluate in parallel every clock; named nets (pij, tkT_i) feed the OR gates on the right.







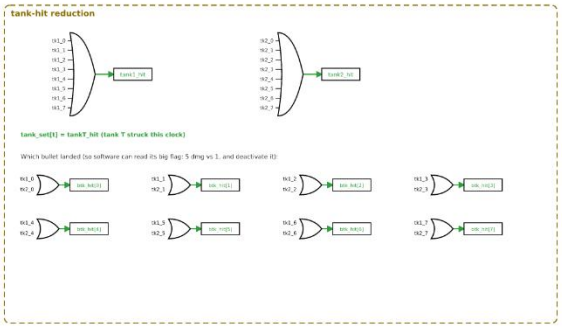
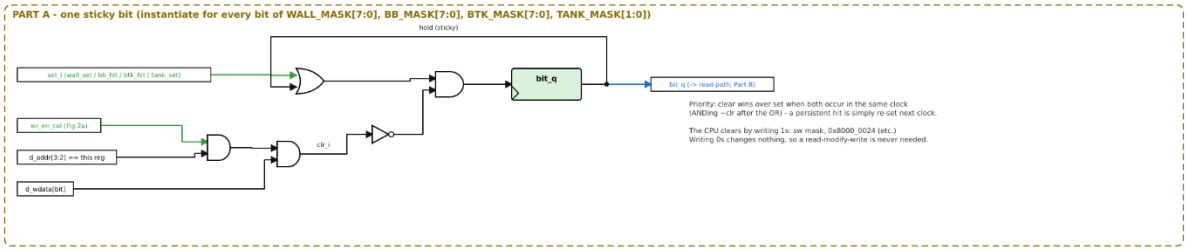


Fig 6d - Collision Engine (3/3): Sticky Hit Registers (write-1-to-clear) and CPU Read Path

Hits are latched until the CPU clears them, so a hit lasting less than one software frame is never missed. Registers 0x20..0x2C, word access.



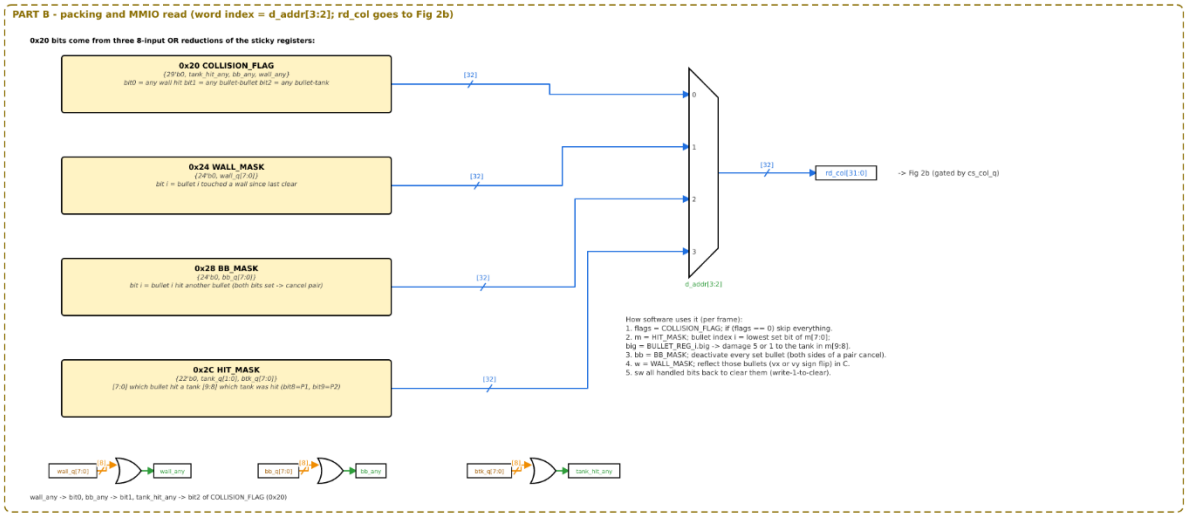
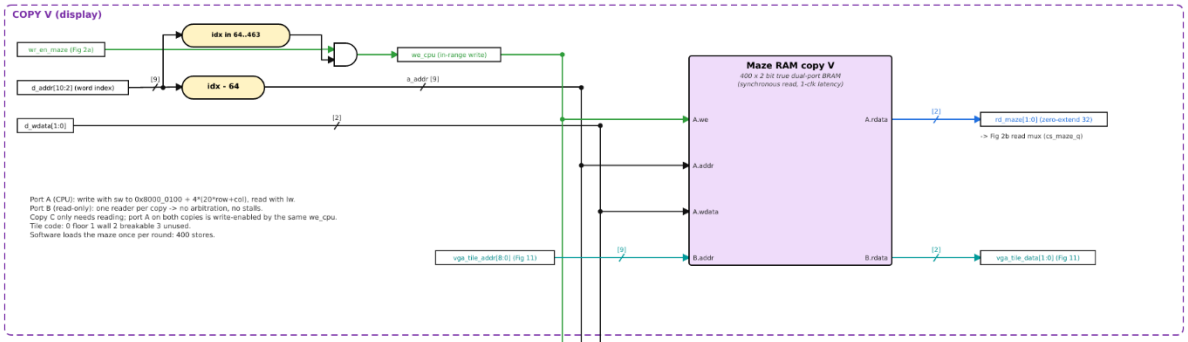


Fig 7 - Maze Tile RAM: Two Mirrored True-Dual-Port BRAMs (20 x 20 tiles, 2 bits each)

FPGA BRAM has only 2 ports, but 3 users need it (CPU, VGA, collision). Fix: two identical RAMs; the CPU writes BOTH with the same data, VGA reads copy V on port B, collision reads copy C on port B, CPU reads back copy V on port A (copy C port A is write-only).



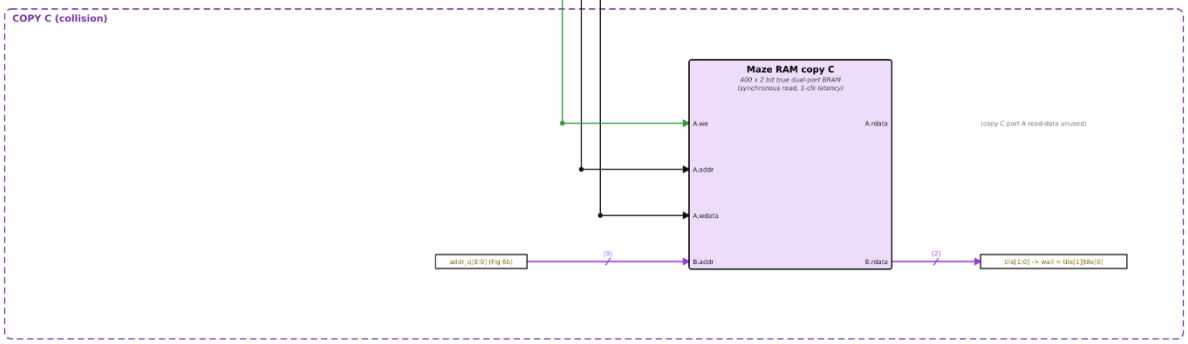


Fig 8 - VGA Timing Generator (640 x 480 @ 60 Hz, 25 MHz pixel enable)

Totals: 800 clocks per line (640 visible + 16 front + 96 sync + 48 back), 525 lines per frame (480 visible + 10 front + 2 sync + 33 back).

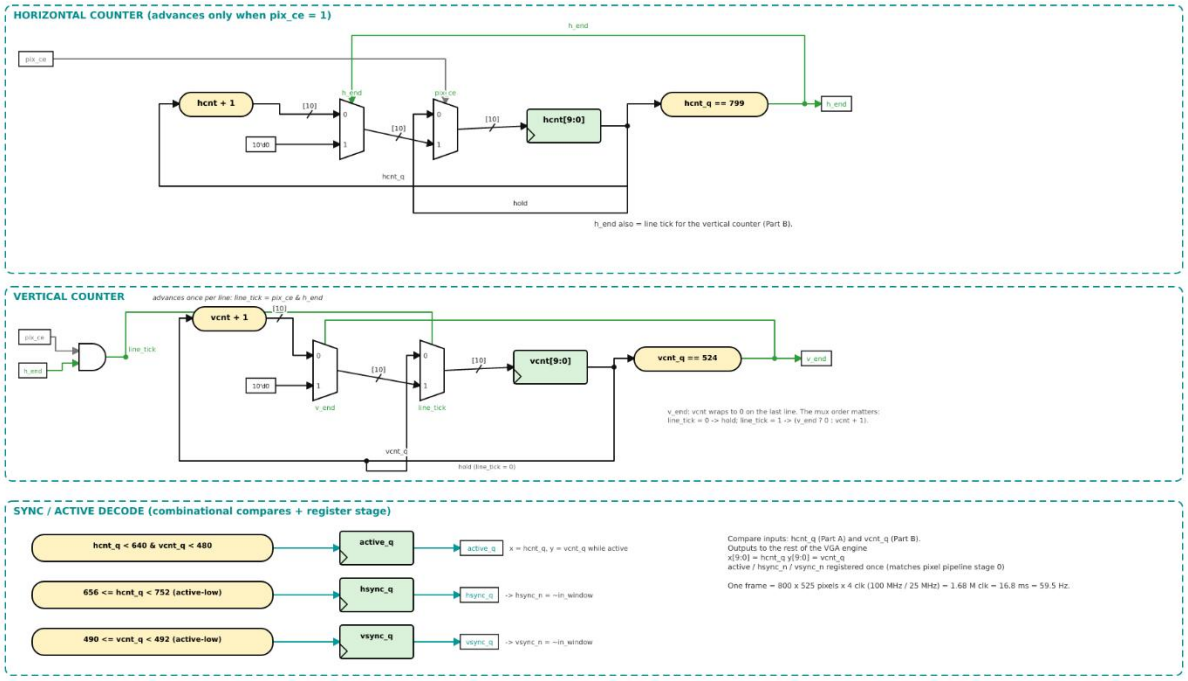
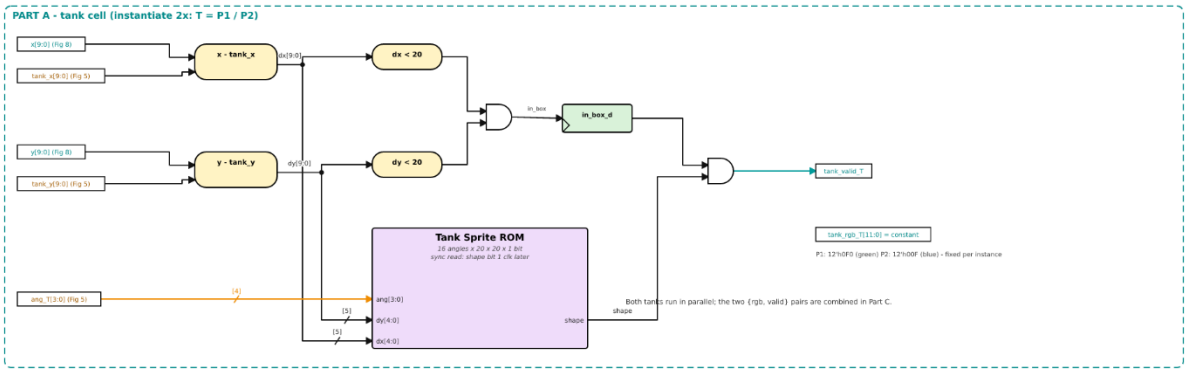
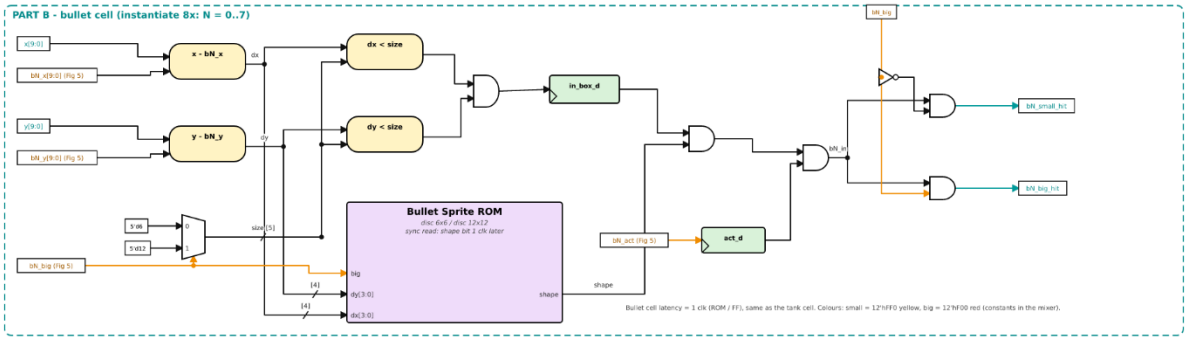


Fig 9 - Sprite Engine: Per-Pixel Tank and Bullet Hit Test (x, y from the VGA timing generator)

offset = (x - sprite_x, y - sprite_y). Unsigned subtract makes 'negative' offsets huge, so ONE compare (offset < size) tests both edges. If inside the box, a small ROM gives the shape bit (1 clk latency, so the in-box flag is delayed 1 FF).





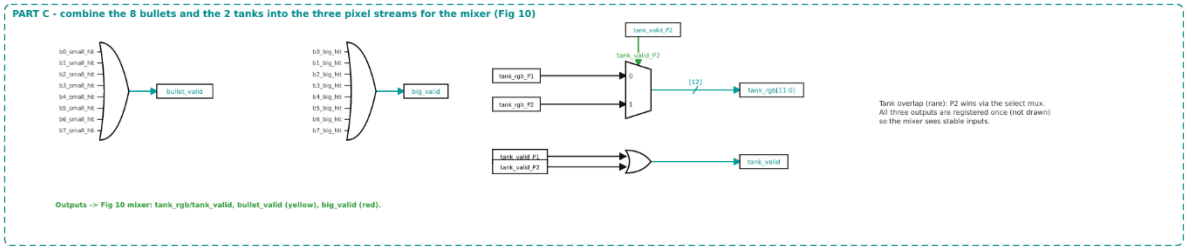


Fig 10 - Pixel Priority Mixer, Sync Delay Line and VGA Output Registers

Priority HUD > Big bullet > Bullet > Tank > Wall > Background (one mux per level, highest priority last = closest to the output). All inputs are aligned to the same pixel (each source has 1-2 clk latency).

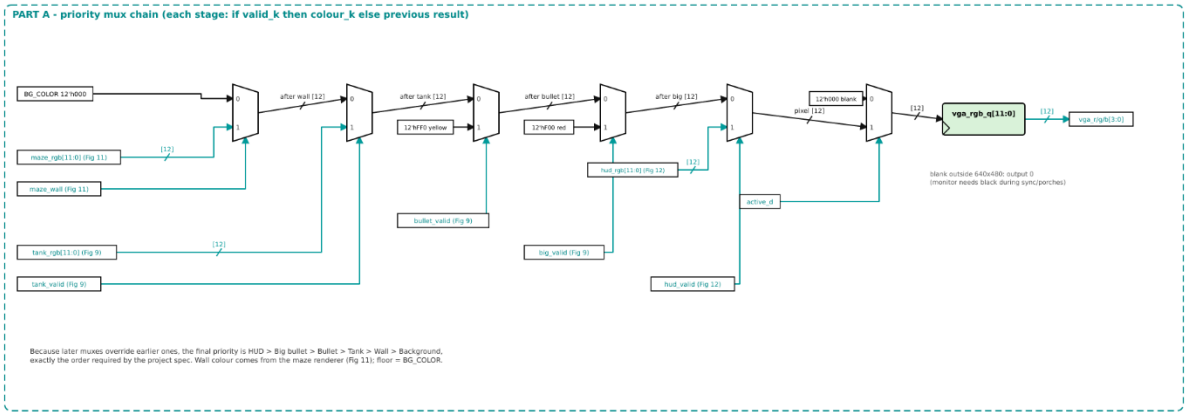


Fig 11 - Maze Tile Renderer: pixel (x, y) -> tile address -> wall / floor colour

24 px tiles, 20 x 20 = 480 x 480 playfield at the left of the 640 x 480 screen. No divider: tile column / row are small counters that step in sync with x / y.

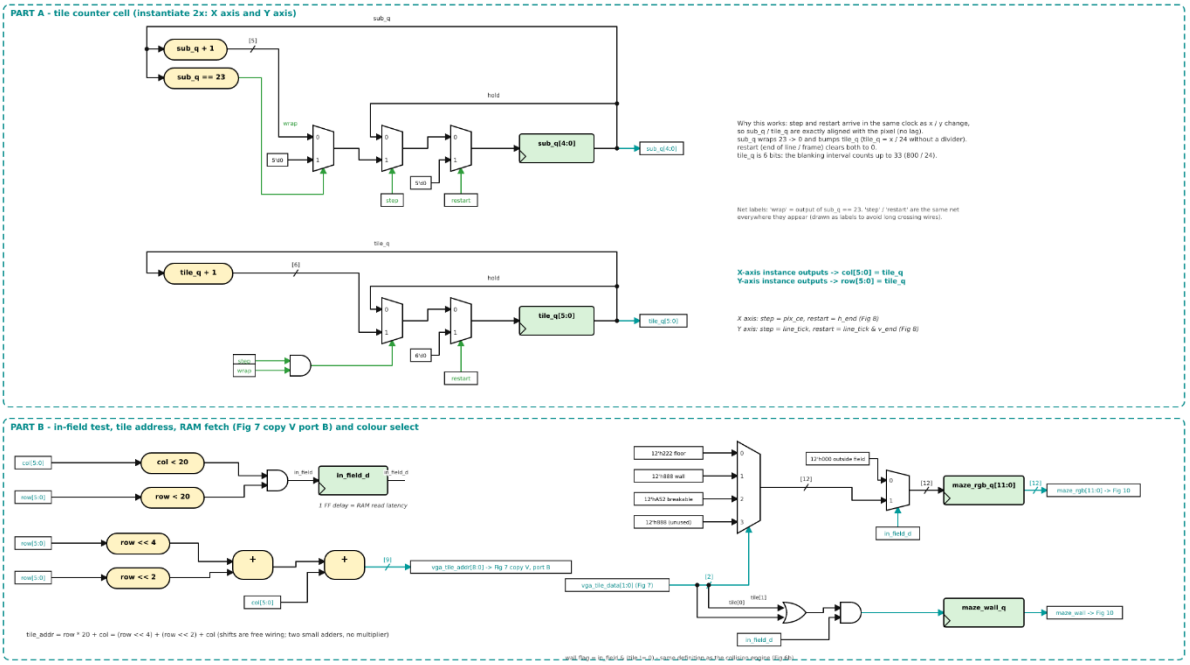


Fig 12 - Health HUD: 2 x 20 hearts in the 160 px strip right of the maze

Hearts are 8 x 8 px, so slot = dx[7:3] and pixel-in-heart = dx[2:0] are pure wiring (no divider). P1 row at y = 32..39, P2 row at y = 432..439, strip x = 480..639.

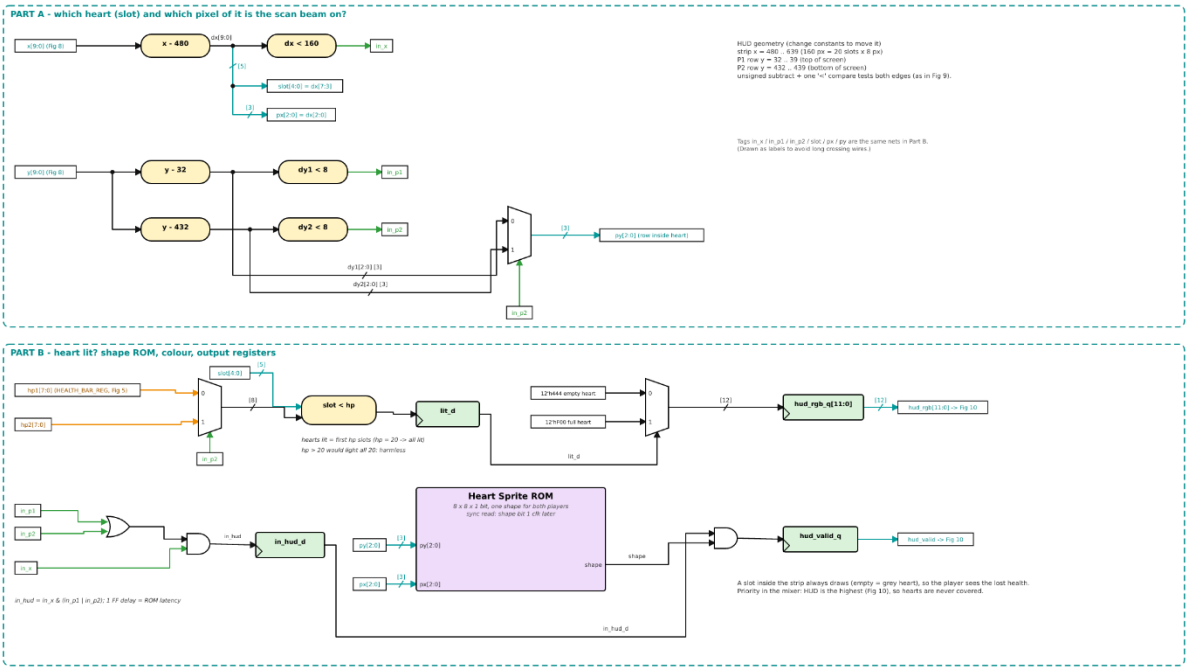
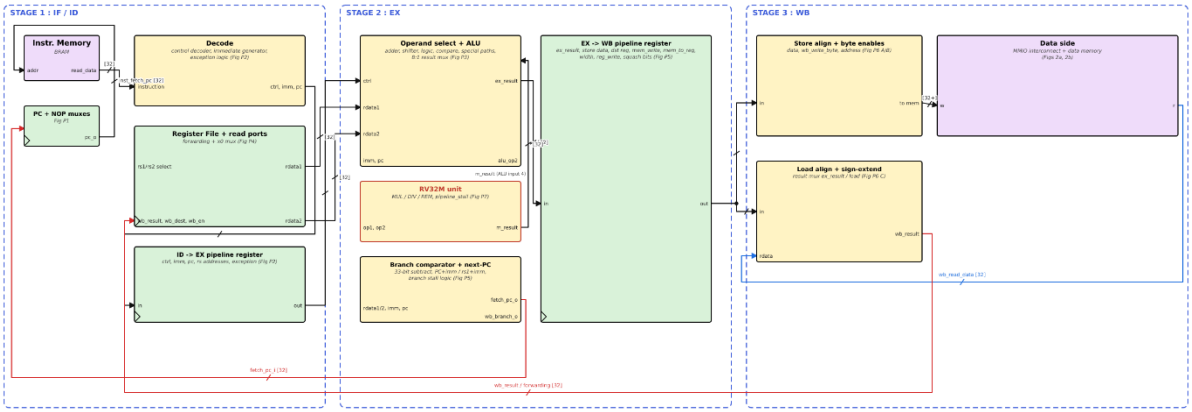


Fig P0 - RV32IM 3-stage pipeline: overview (how Figs P1 - P7 connect)

IF/ID -> EX -> WB. Memories: instruction BRAM, data side = MMIO interconnect (Figs 2a / 2b). Red wires: write-back forwarding / branch redirect.

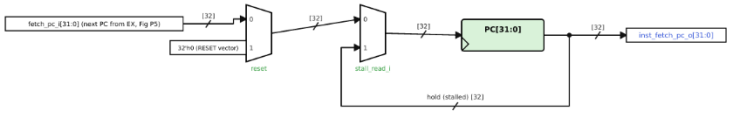


stall_read_i (data-memory read in flight) and pipeline_stall (M unit) freeze the PC / IF-ID registers; wb_stall_first / second squash wrong-path instructions after a taken branch (Fig P1).

Fig P1 - Pipeline Stage 1 (IF): PC register, instruction NOP muxes, branch-flush shift register

Drawn from the team's IF/ID sketches. Where a sketch was ambiguous the choice is noted in italics - check it against your RTL.

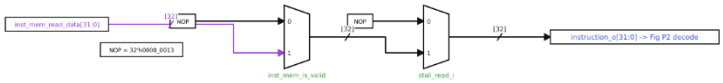
PART A - PC register (reset mux, stall-hold mux, FF) -> inst_fetch_pc_o



inst_fetch_pc_o -> Instruction Memory address and -> ID/EX register (pc_ex, Fig P2)

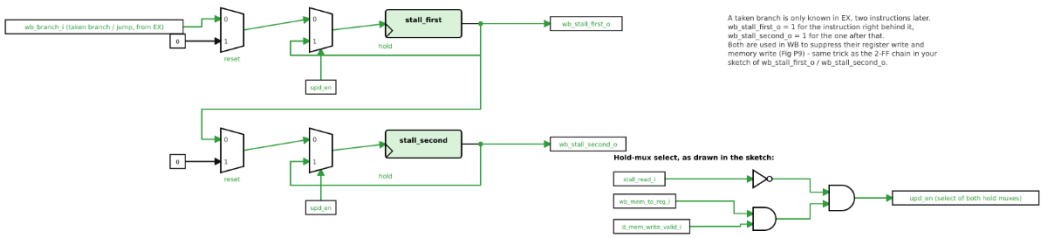
reset = 1 -> PC = 0. stall_read_i = 1 (data-memory read in flight) -> PC holds. Otherwise PC <= fetch_pc_i = (branch ? target : PC + 4) computed in EX.

PART B - instruction path: two NOP-insert muxes (as in the sketch)



Muxes as in the sketch: input 0 = NOP, input 1 = instruction; select 1 = inst_mem_is_valid, then stall_read_i.
NOP = addi x0,x0,0 (32'h00000013). The polarity of the second select (stall_read_i) is read from the sketch - verify it against your RTL.

PART C - branch flush shift register (squash the two instructions fetched behind a taken branch)



A taken branch is only known in EX, two instructions later. wb_stall_first_o = 1 for the instruction right behind it, wb_stall_second_o = 1 for the one after that. Both are used in WB to suppress their register write and memory write (Fig P9) - same trick as the 2-FF chain in your sketch of wb_stall_first_o / wb_stall_second_o.

upd_en = ~stall_read_i & (wb_mem_to_reg_i & d_mem_write_valid_i) -> select = 1 loads the new value, 0 holds. The AND-gate inputs are read from the sketch - verify them against your RTL.

Fig P2 - Pipeline Stage 1 (ID): immediate generator, decoder, exception logic

Follows your 'I-D stage' immediate-mux notebook page and the ID/EX block sketch. RV32M adds one decode line (funct7 = 0000001, red).

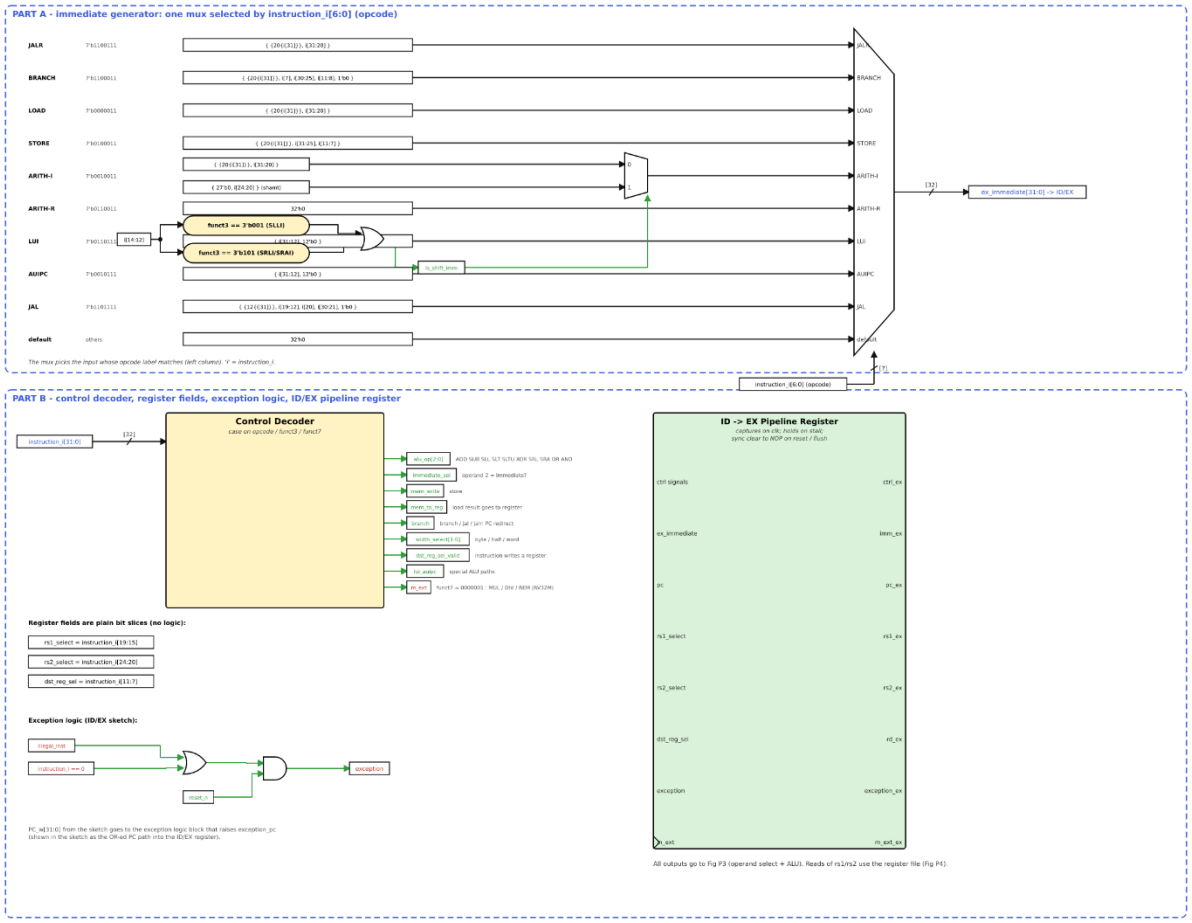


Fig P3 - Pipeline Stage 2 (EX): operand select and ALU core with 8:1 result mux

From your 'ALU Operand Selection' and 'ALU Core + Supported Operations' sketches. Operand 1 needs no mux; operand 2 = register or immediate.

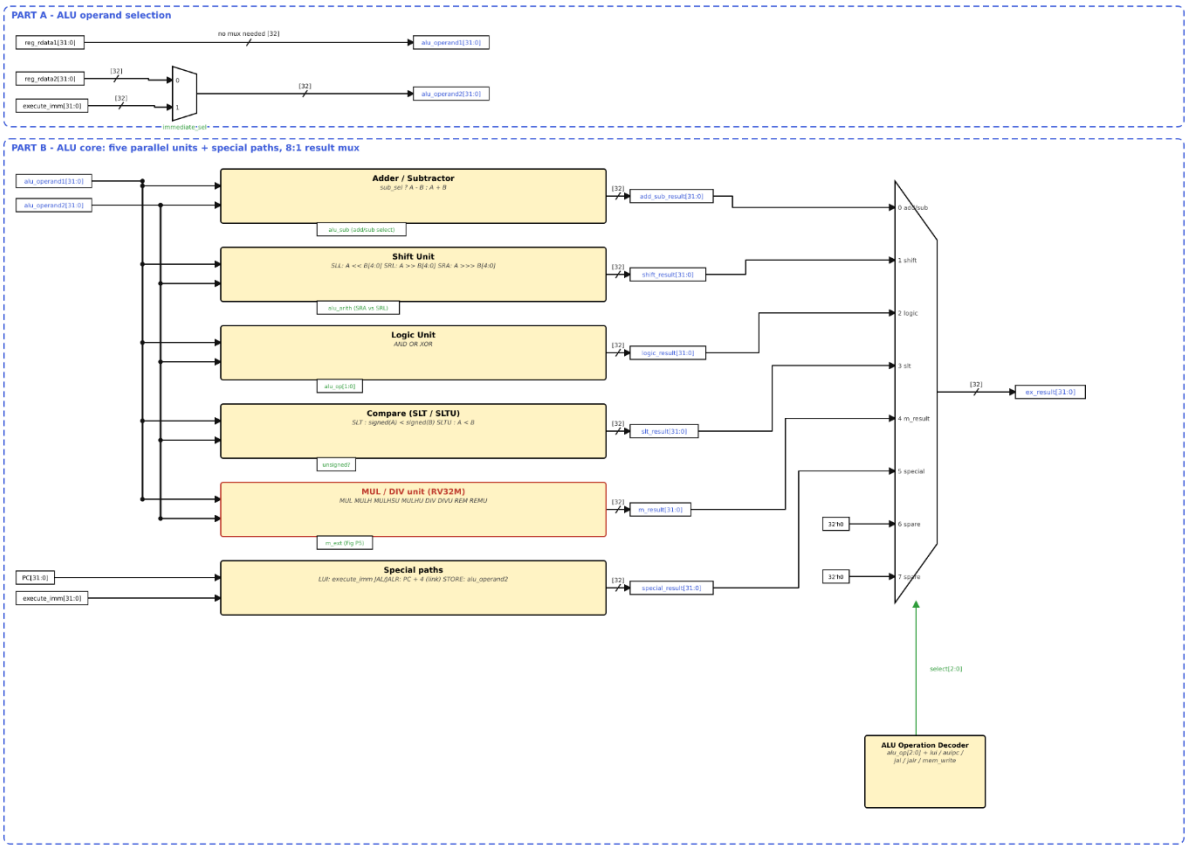
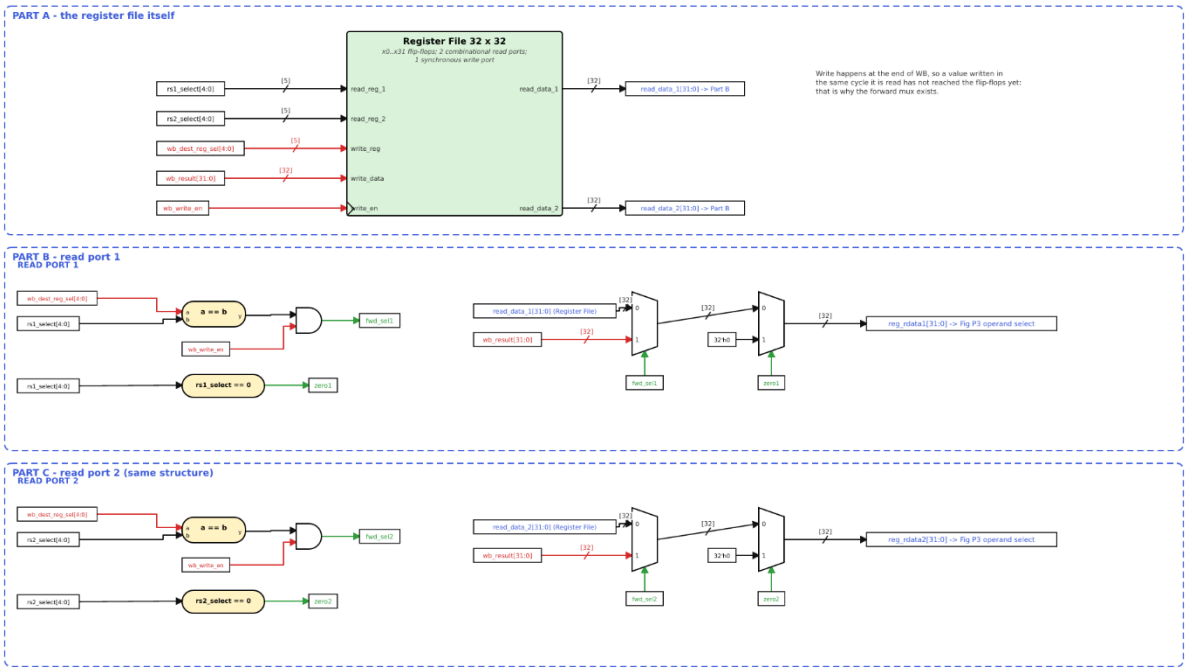


Fig P4 - Register File and read-port logic: write-back forwarding + x0 forced to zero

Each read port = register-file output -> forward mux (take wb_result if the same register is being written back) -> x0 mux. Select polarity read from your pipeline sketch: verify.

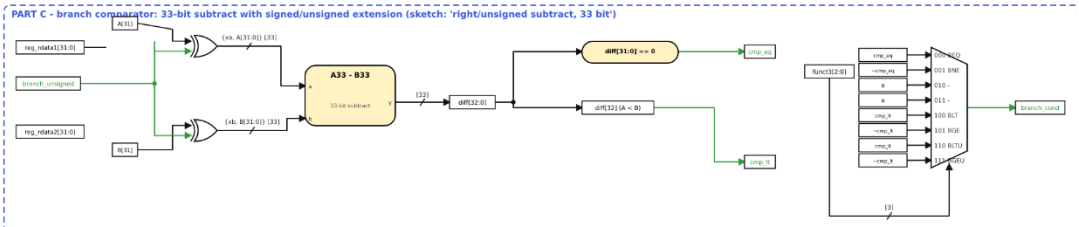


Red wires = write-back signals coming from the WB stage (Fig P6). Green = select signals.

Fig P5 - EX stage: branch comparator, next-PC logic, branch-stall logic and EX/WB register

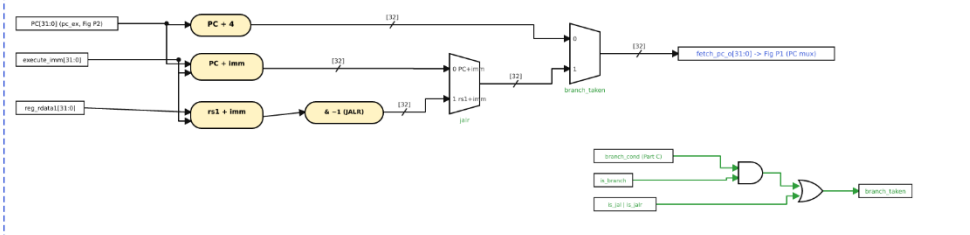
From your 'High-level EX stage block diagram': C = branch comparator, D = next-PC logic, E = branch stall logic, F = EX/WB register. (A, B = Fig P3.)

PART C – branch comparator: 33-bit subtract with signed/unsigned extension (sketch: 'right/unsigned subtract, 33 bit')



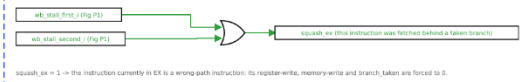
The unsigned bit flips the sign extension so a single 33-bit subtract gives the right 'less than' for both signed and unsigned compares.

PART D – next-PC logic (fetch_pc_o = redirect ? target : PC + 4)



branch_taken = (branch_cond & is_branch) | jal | jalr -> also drives wb_branch_o (Part E) and the flush shift register (Fig P1).

PART E / F - branch stall logic and the EX -> WB pipeline register



squash_ex = 1 -> the instruction currently in EX is a wrong-path instruction: its register-write, memory-write and branch_taken are forced to 0.



Outputs go to Fig P6 (WB / load-store align).

Fig P6 - WB stage: store byte-lane alignment, byte enables, load alignment / sign-extension, result mux

From your 'wb_write_data / wb_write_byte' mux sketch. SB / SH / SW decide how data and the 4 byte enables are placed in the 32-bit word.

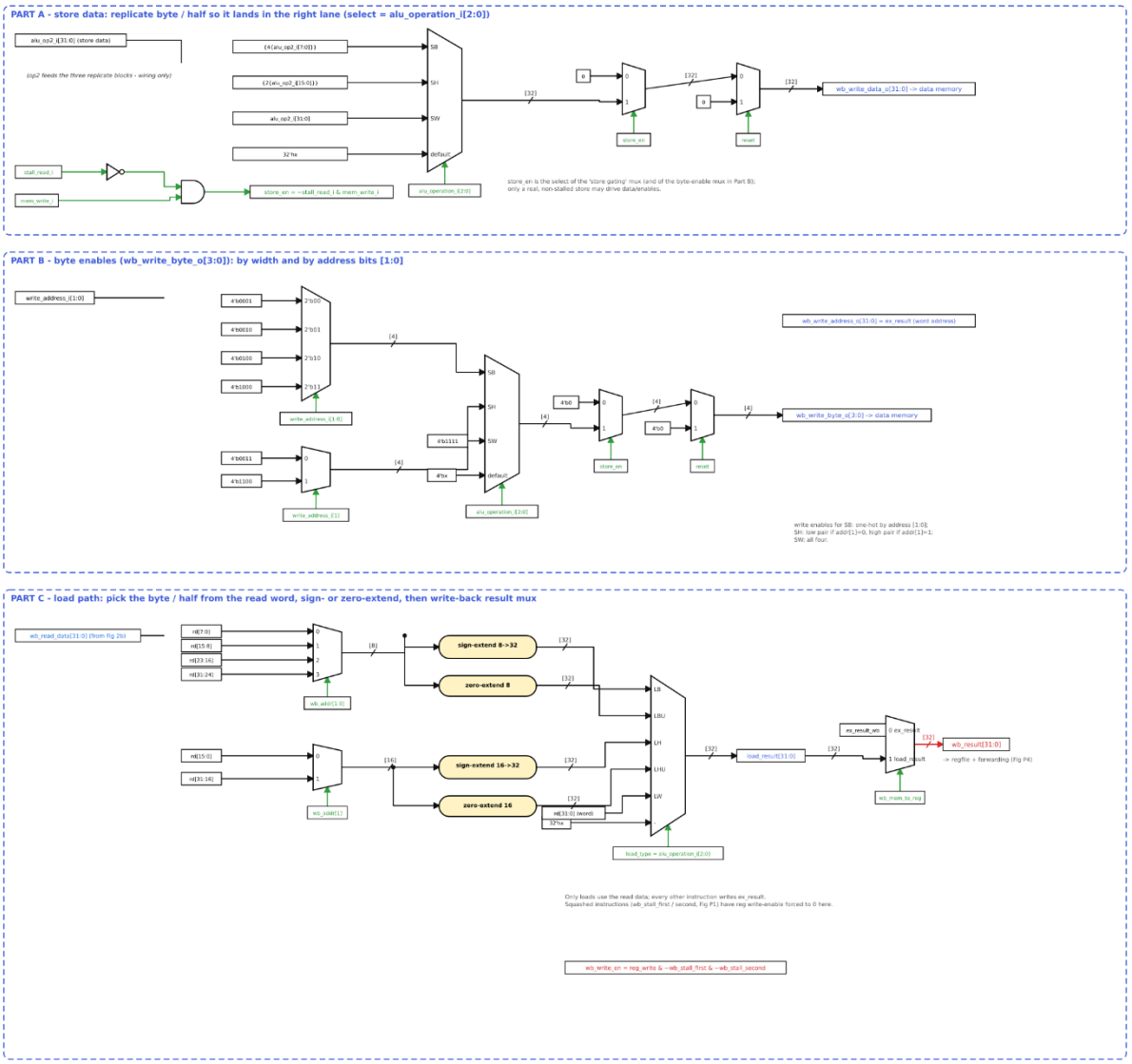


Fig P7a - RV32M (1/2): multiplier unit (MUL / MULH / MULHSU / MULHU)

One 33 x 33 signed multiplier covers all four variants: bit 32 of each operand = operand[31] AND 'operand is signed'. Maps onto DSP48 slices.

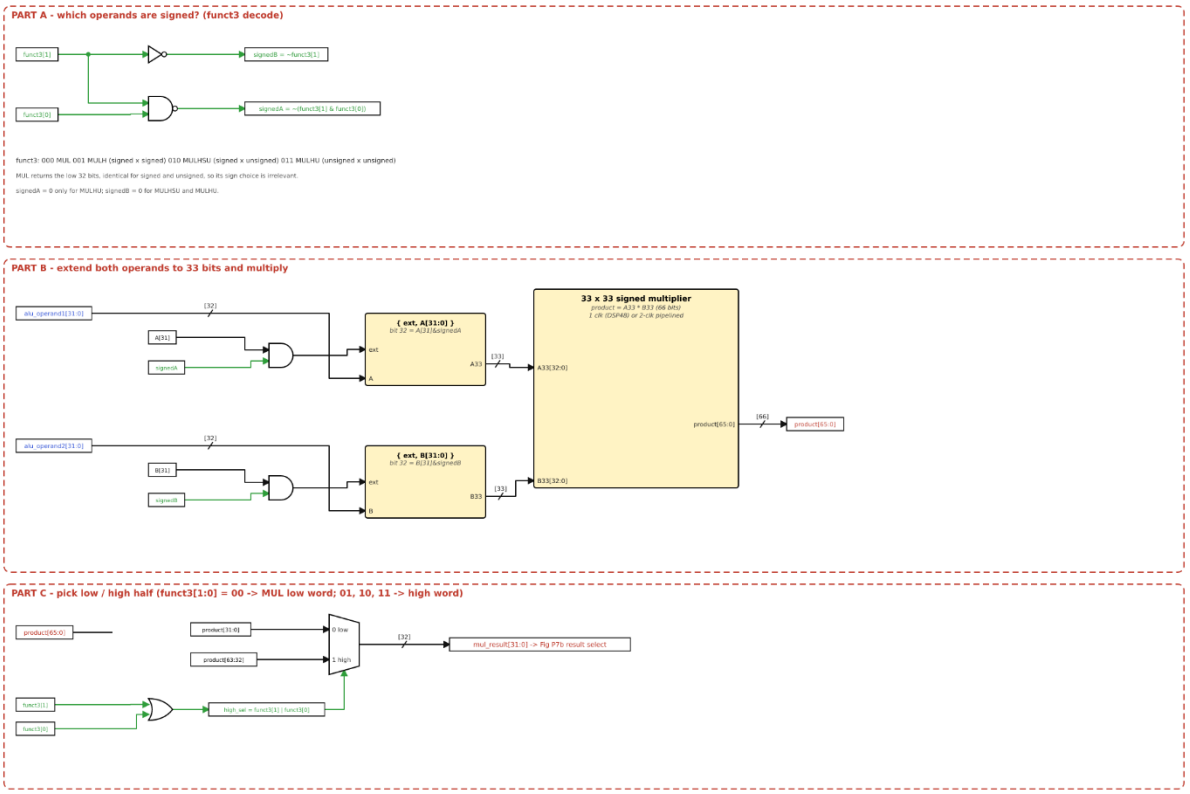
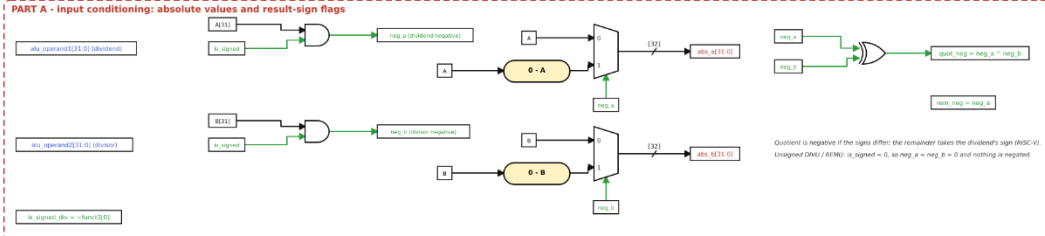


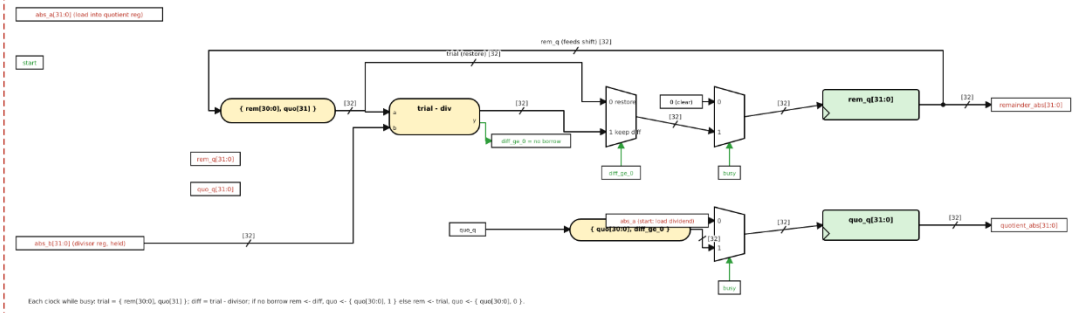
Fig P7b - RV32M (2/2): iterative divider (DIV / DIVU / REM / REMU), special cases and pipeline stall

Radix-2 restoring divider on magnitudes: 32 clocks. Signs are fixed before / after. Divide-by-zero and overflow follow the RISC-V spec (no trap).

PART A - input conditioning: absolute values and result-sign flags



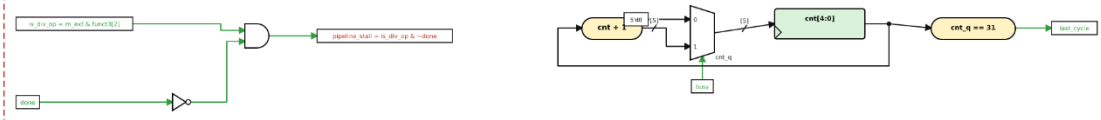
PART B - the 32-clock restoring divider: remainder / quotient shift registers, subtract, restore, counter



Each clock while busy: trial = { rem[30:0], quo[31] }; diff = trial - divisor; if no borrow rem <- diff, quo <- { quo[30:0], 1 } else rem <- trial, quo <- { quo[30:0], 0 }.

(The mux inputs 'restore' / 'keep diff' implement the restoring step; start loads abs_a into the quotient register and clears the remainder.)

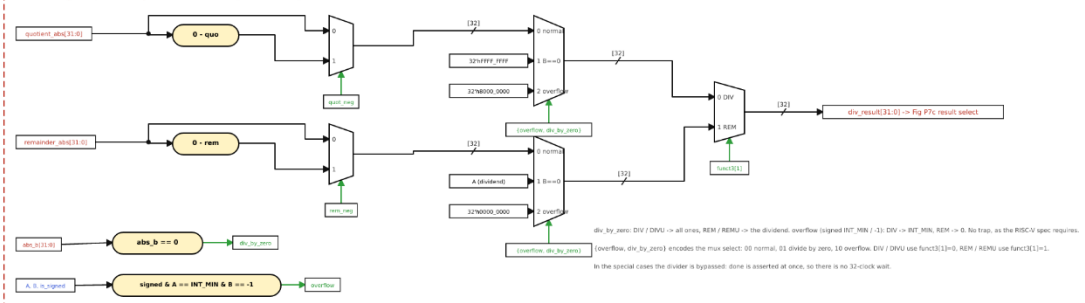
PART C - control: IDLE / BUSY state, 5-bit counter, stall



pipeline_stall = 1 freezes IF/ID (PC hold, Fig P3) and inserts NOP bubbles toward WB until done = 1; the result is then written once.

busy: set by 'start' (first cycle of a DIV/REM in EX), cleared by last_cycle. done = last_cycle (registered) or special-case bypass.

PART D - sign fix, divide-by-zero / overflow special cases, final selection



div_by_zero: DIV / DIVU -> all ones, REM / REMU -> the dividend. overflow (signed INT_MIN / -1): DIV -> INT_MIN, REM -> 0. No trap, as the RISC-V spec requires.

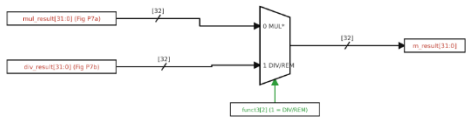
{overflow, div_by_zero} encodes the mux select: 00 normal, 01 divide by zero, 10 overflow. DIV / DIVU use funct3[1]=0, REM / REMU use funct3[1]=1.

In the special cases the divider is bypassed: done is asserted at once, so there is no 32-clock wait.

Fig P7c - RV32M integration: result select into the ALU and pipeline stall / bubble control

The M unit sits in EX beside the ALU. Its result enters input 4 of the ALU result mux (Fig P3); a DIV/REM holds the pipeline until done.

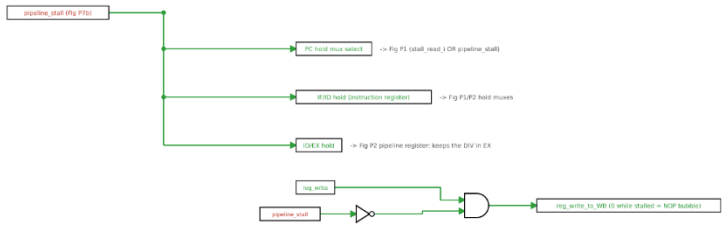
PART A - M-unit result select and ALU result-mux input 4



m_result -> input 4 of the 8:1 ALU result mux (Fig P3).
ALU select = 3'b100 when m_ext = 1.

funct3 map: 000 MUL 001 MULH 010 MULHSU 011 MULHU | 100 DIV 101 DIVU 110 REM 111 REMU

PART B - stall: hold IF/ID, bubble into EX/WB until done



While the divider runs, the DIV stays in EX and the instruction behind it is held in ID: WB sees a bubble (writes disabled) so nothing wrong is committed.